

Home Problem 2

Stochastic Optimization Algorithms
Daniel Peña Fonseca
20031020-T330

Problem 2.1.

Description

For this exercise, we want to study the traveling salesman problem. Here, we want to visit 50 cities and return to the starting city, and we want the path to be as short as possible. To determine the shortest path, we implement an Ant system.

Parameters: In this case, we initialize the pheromone levels with $\tau_{ij} = \tau_0 = 0.1$, $\alpha = 1$, $\beta = 5$ and an evaporation rate of $\rho = 0.5$. We also set a minimum pheromone level of $\tau_{min} = 10^{-15}$.

The first parameters appear in the expression for the probability that an individual who has followed a trajectory S goes from the city i to city j :

$$p(e_{ij}|S) = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{\nu \notin L_T(S)} \tau_{ij}^\alpha \eta_{ij}^\beta} \quad (1)$$

Whereas the evaporation rate, ρ , allows us to reduce the impact of previous pheromone levels when updating these:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \Delta\tau_{ij} \quad (2)$$

Results: Using a population of 50 individuals and the target path length specified in the skeleton file, the length of the shortest path found was $L \approx 99.0005$. This track is presented in Figure 1.

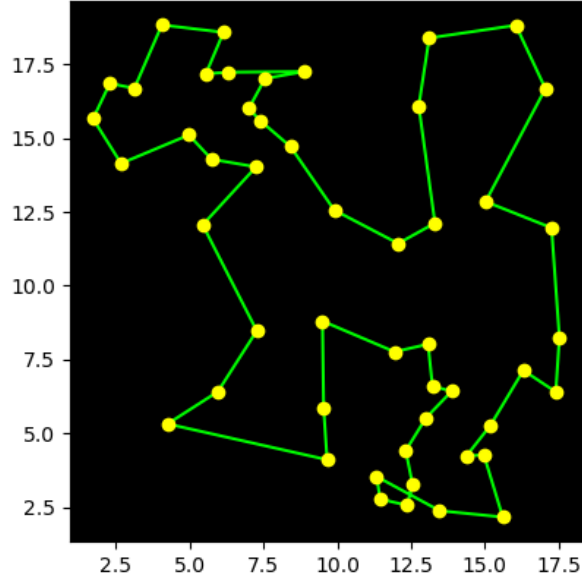


Figure 1: Plot of the shortest path found for the traveling salesman problem using an Ant system. This path has a length of $L \approx 99.0005$.

Notes on the code: The file `run_ant_system.py`, prints every new best path along with its length, and shows the plot of the track. When run with the seed provided in the code, we obtain the plot in Figure 1 as the final path. The track presented in the figure was manually stored in `best_result_found.py`.

Problem 2.2

In this case, we want to obtain the minima of a function using Particle Swarm Optimization. To do so, we first define our parameters: 30 particles, 500 time steps of size $\delta t = 0.1$, and a maximum velocity of $v_{max} = 5$. Other fixed parameters are: scaling factor $\alpha = 1$, step $\Delta t = 1$ and an initial inertia weight of $w = 1.4$ with a decaying factor of $\beta = 0.99$. We also set a threshold of 10^{-12} to determine when to stop the search of a minimum: if in the past 10 steps the lowest function value found has not decreased by the threshold, we consider that the system has converged and we stop the search.

In order to detect all the minima in the function, the search process was repeated 200 times. The observed minima are presented in Table 1 and Figure 2.

l	x_1^*	x_2^*	$f(x_1^*, x_2^*)$
1	2.9999	2.0000	$5.3925 \cdot 10^{-15}$
2	3.5844	-1.8481	$2.134 \cdot 10^{-12}$
3	-3.7793	-3.2831	$2.0833 \cdot 10^{-14}$
4	-2.8051	3.1313	$7.6032 \cdot 10^{-14}$

Table 1: To obtain the minimum values of the function $f(x_1, x_2)$, we used the PSO algorithm, and found that these minima appeared at the different $\mathbf{x}^* = (x_1^*, x_2^*)$ presented. When evaluated in these points, the function took the values $f(x_1^*, x_2^*)$. The index l just enumerates the minima.

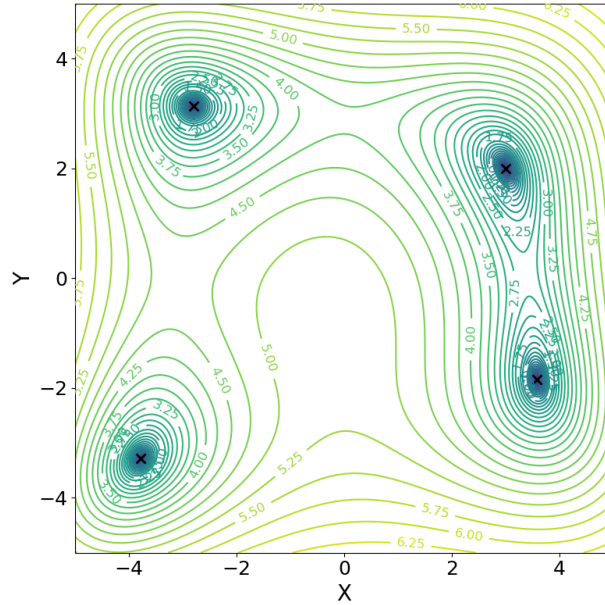


Figure 2: Contour plot of the function $g(x) = \log(a + f(x_1, x_2))$, with $a = 0.01$. The minima presented in Table 1 are represented in the figure with black crosses.

In the given code, we assume that the different minima (x_i^*, y_i^*) are separated at least by a distance of 0.01. With this assumption we (nearly) ensure that when running the 200 search processes, the program only outputs different minima; instead of outputting the same ones multiple times.

Notes on the code: For Problem 2.2, there is only one file: `run_pso.py`. When run, the minima are printed and the plot presented in Figure 2 is shown. The exact minima presented in Table 1 and Figure 2 are obtained with the seed provided in the code.

Problem 2.3

In this case, we have a model describing how the speed of a truck evolves in time. Velocity change depends on multiple elements: some constants, such as the mass of the truck or the ambient temperature, among others; and some variables, like the engine temperature, T ; the slope of the

current position, $\alpha(x)$; the pedal pressure, P_p and the current gear, g . The goal is to design a feed forward neural network (FFNN) that, given the speed, the position, and the angle of the current state, it returns the optimal pedal pressure and gear change.

Since we are working with **evolutionary algorithms**, we will use evaluation, selection, crossover and mutation to search for the best neural network; i.e. the set of weights returning the optimal solution. Since all these steps are easier when working with chromosomes, especially if these are simply lists of elements, we will work with these objects and just convert (decode) them to the weight matrices when we want to evaluate the performance of the truck in a certain slope.

For this exercise, we choose $n_h = 4$ hidden neurons with $w_{max} = 10$. In terms of the evolution of the system, we perform selection through tournaments, with a tournament size of 4 individuals (chromosomes), and a tournament probability of 0.8. For the offspring, we use one-point crossover with a crossover probability of 0.7, and mutation with a constant probability of $1/N$, where N denotes the number of genes, and is given by the size of the matrices: $N = n_i \cdot (n_h + 1) + n_o \cdot (n_h + 1)$, with $n_i = 3$, $n_o = 2$ indicating the number of inputs and the number of outputs, respectively. We will perform 100 generations, given a population size of 30 chromosomes.

The **truck model** with the equations provided has been defined in a class called *truck_model.class* inside the file *truck_model.py*. This object will be called to update the state of the truck variables and to update the velocity and temperature.

To **evaluate** how well a truck performs under a given neural network for a certain slope, we simulate the evolution of the truck. To do so, we initialize at position $x = 0$ m, speed $v = 20$ m/s, temperature $T = 500$ K, pedal pressure $P_p = 0$ and seventh gear $g = 7$, and in each step, we calculate (using the last velocity) the position, the angle and the temperature, that we feed to the FFNN to obtain g and P_p . With these elements, we calculate the speed that the truck will use to travel to its next point.

After completing (or failing) a slope i , we compute the **fitness** value of the network in the slope, f_i using the expression:

$$f_i = d_i \cdot \langle v_i \rangle \quad (3)$$

Where d_i denotes the distance traveled in the slope: $d_i \leq L$; and $\langle v_i \rangle$ indicates the average speed of the truck during the traveled distance. With this fitness expression, we seek to penalize both trajectories that don't reach the total distance $x = L$ and slow trajectories $\langle v_i \rangle \ll v_{max}$. Given this fitness function, the best fitness we can get (theoretical) is $1000\text{m} \cdot 25\text{m/s} = 25 \cdot 10^3 \text{m}^2/\text{s} \equiv 25 \cdot 10^3 \text{ Js/kg}$.

For the training and validation set, we want to measure the fitness score of a certain network in the whole set, not only for one slope. To address this, we define the fitness value of a network in a set S of slopes, as the average of the fitness in the different slopes. I.e., in a set S with M slopes, the fitness value of the network in the set, $F_{i,S}$, satisfies:

$$F_{i,S} = \frac{\sum_i^M f_i}{M} \quad (4)$$

Note: To maximize precision in d_i , we must consider that the maximum distance traveled is $L = 1000$ meters. Therefore, when, in our last step, we have gone from point O ($O < L$) to point P , where P is past L , we should just compute the time it took the truck to go from O to L . This time

is calculated using the velocity (updated) in point O , $v_{O \rightarrow P}$:

$$\delta t = \frac{L - O}{v_{O \rightarrow P}} \quad (5)$$

Adding δt to the time it takes the truck to reach point O , t_O , we compute how much time the truck needs to go from $x = 0$ to $x = L$: $t_L = t_O + \delta t$. In these cases, the average speed satisfies $\langle v_i \rangle = x_L / t_L$.

To train our network, obtain the best set of weights, and test its validity, we use we use 10 training slopes, 5 validation slopes, and a test set of 5 slopes. With these slopes having the following shape:

$$a \pm b \sin(dx/f) \pm c \cos(ex/g) \quad (6)$$

Here, the parameters a, b, c, d, e, f and g (not related to the gear) are randomly selected from a list of values similar to those provided in the skeleton file. Whether we sum or subtract the sine and/or cosine is also randomized.

Randomizing the parameters rather than manually setting them eliminates any bias and generally leads to better results. Also, the larger the training set, the more function behaviors will be already *known* by the network, leading to better performance in the validation and test sets.

After taking all these factors into account, we can first train the network. Here, we train during 100 generations, and in each step we store the best fitness scored by an individual from the population in the training and in the validation set. Figure 3 presents these results.

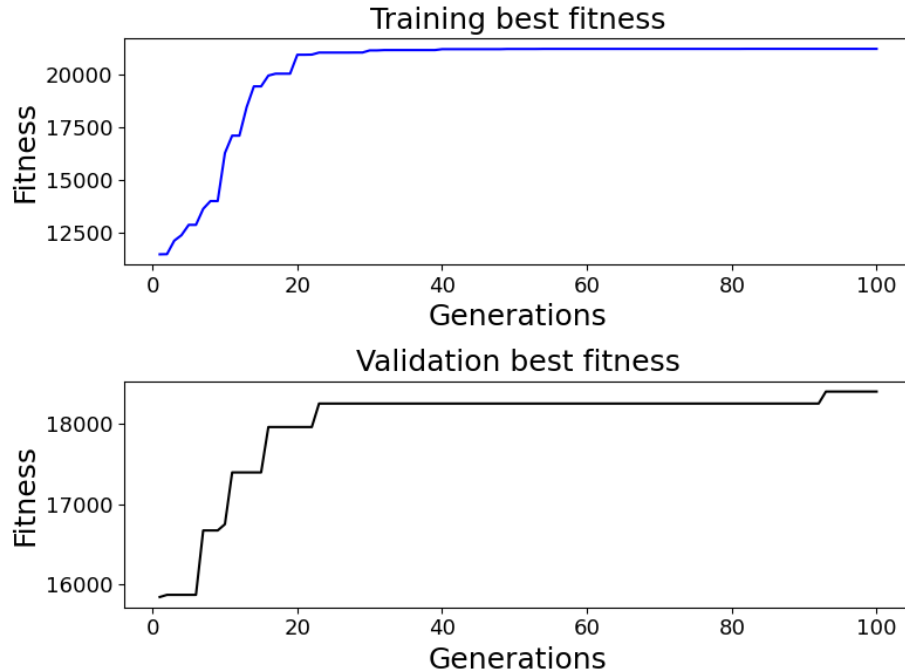


Figure 3: Representation of the all time best fitness value as a function of the generation. The results are presented for the training set (above) and for the validation set (below).

Figure 3 shows how fitness values are higher for the training slopes than for the validation set. This is an expected result, since in the population, we enhance the presence of individuals with higher fitness values (tournament probability, $p_{tour} > 0.5$) in the training set, not in the validation slopes. However, the best fitness for the validation set increases with the generations, as individuals (networks) in the population learn better the characteristics of the training curves; and given their random nature, the networks are most likely learning general behaviors, leading to better performance in unknown curves.

Following this procedure with different seeds, we find an average success of around 80%. That is, when testing the network with the test set, we usually get fitness values greater than $20 \cdot 10^3 \text{ m}^2/\text{s}$ on 4 of the 5 slopes, with these 4 slopes being completed (truck reaching $x = L$). Given the small size of the training set, these results seem satisfactory.

Figure 4 shows a test performed on the best network found in all the generations for the same seed used to generate Figure 3. During the test, we measure the evolution of the angle, the pedal pressure, the gear, the velocity and the temperature as a function of the distance. The fitness value obtained for the test slope is also presented.

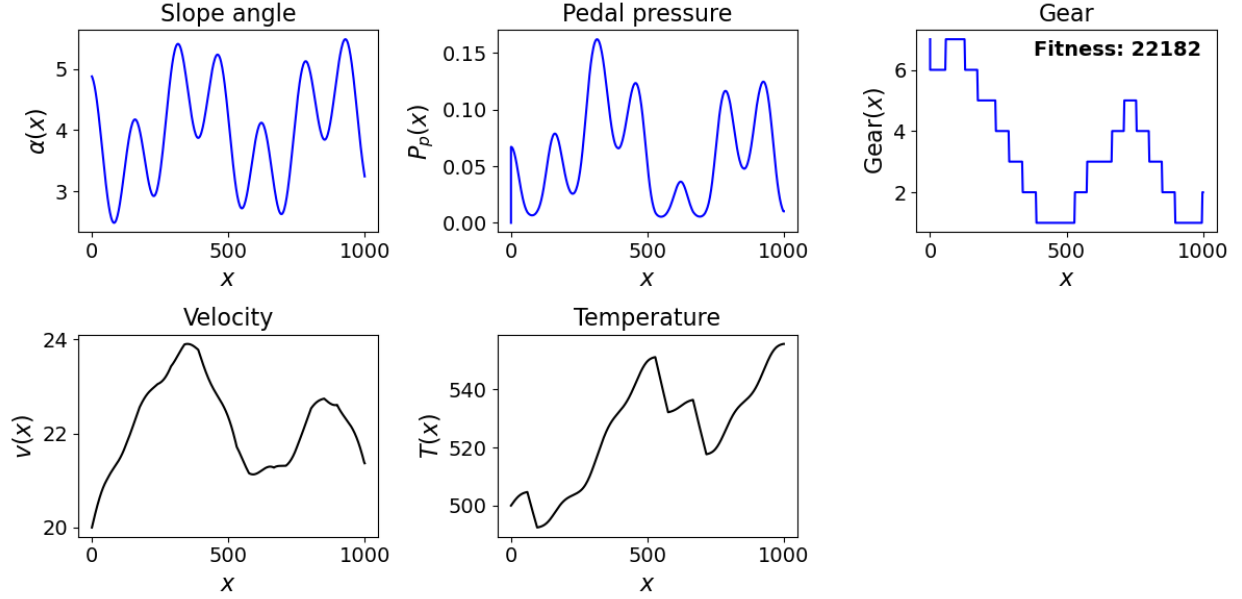


Figure 4: Representation of how different variables in the system evolve with the distance, x . These variables are: angle (top left), pedal pressure (top center), gear (top right), velocity (lower left) and temperature (lower center). The fitness achieved by the network in this particular slope is $f \approx 22182 \text{ m}^2/\text{s}$.

Notes on the code: The coding and encoding functions are defined in the file *run_encoding_decoding_test.py*. The truck model used to calculate the velocity is stored in a class in the file *truck_model.py*. The file *slopes.py* contains the slopes used for training, validation and test, including the seed used to randomize the parameters of each function. The program *run_ffnn_optimization.py*, when run, performs the training and saves the best chromosome in the file *best_chromosome.py*. When run, *run_ffnn_optimization.py* also prints, for each generation, the best fitness for the training and

the validation set, and saves these values in the file *data_fitness_evolution.py*. To plot the fitness evolution (Figure 3), we run the file *fitness_plot.py*. Finally, to test our best network (stored in *best_chromosome.py*) we run the file *run_test.py*. If the user wants to test the network for another set of slopes, change the variable *data_index* to 1 for the training set, 2 for the validation set, and 3 for the test set. The slope index is given by the variable *slope_index*, ranging from 1 to 10 for the training set, and from 1 to 5 for the validation and test set. The results presented in Figure 3 and Figure 4 are obtained using the seeds already provided in the files *slopes.py* and *run_ffnn_optimization.py*. Figure 4 specifically, is obtained using the third slope of the test set.

Problem 2.4

Description of LGP

First, we **initialize** the population with 25 instructions, each instruction with 4 elements, called genes; therefore, we initialize with 100 genes per chromosome. Second, we enter the loop of generations and start by **evaluating** the population so that we measure the fitness of all the individuals (chromosomes). After storing the best chromosome, along with its fitness value, we go through the **selection** process using tournament selection with a tournament size of 5 individuals, and a tournament probability of 0.7. Now we do two-point **crossover** with a crossover probability of 0.8; here, we set a length limit of 400 genes (100 instructions), and we set a penalty for those chromosomes with a length greater than 300 genes (75 instructions). The expression for the penalty has been arbitrarily selected; if we define F_F as the final fitness (after the penalty), L_c as the length of the chromosome (number of instruction), and O_F as the fitness value without penalty, we get:

$$F_F = \frac{75}{L_c} \cdot O_F \quad (7)$$

Next, we **mutate**. In a standard evolutionary algorithm, we might fix the mutation probability at $p_{mut} = 1/N$, where N denotes the number of genes per chromosome in the initial population. However, in this case, trying to avoid premature convergence, we set a varying mutation probability. To do so, we consider three possible scenarios:

1. In the last generations, the best fitness of the population has increased significantly.
2. In the last generations, the best fitness of the population has suffered nearly no improvement.
3. In the last generations, the best fitness of the population has suffered a slight improvement.

Since mutation increases variety in the population, we are interested in having some generations where diversity increases significantly, so that we rapidly explore a wide range of the state space (space of all possible chromosomes). Trying to enhance diversity only in certain steps of the evolution, we consider scenario number 1. In these cases, we increase the probability of mutation by a factor 1.2: $p_{mut}(g) = 1.2 \cdot p_{mut}(g - 1)$, where $p_{mut}(g)$ denotes the probability of mutation for the current generation, g , and $p_{mut}(g - 1)$ the mutation probability we had in the previous generation. We set a window size of 5 generation steps, so that, if during the last 5 generations, there has been an improvement greater than 10^{-6} in the best fitness value, we increase the mutation

probability by a factor 1.2. With this window size, we ensure that, during at least 5 generations, we significantly increase diversity. To avoid reaching a state with a mutation probability of 1, as could occur if the system suffers multiple improvements consecutively, we could set a maximum probability of, for example, 0.1; however, this was not needed for this experiment.

In addition to a noticeable increase in diversity for certain steps of the evolution, we have decided to slightly enhance mutation when there is nearly no improvement in the system; i.e. an improvement in the best fitness during the last 5 generations smaller than 10^{-12} . For these cases, we set the mutation probability to $p_{mut} = 1.5/N$.

Finally, when the system undergoes slight improvements, between 10^{-12} and 10^{-6} increase in the best fitness value, we keep the standard probability of mutation $p_{mut} = 1/N$.

Results

Out of different runs, the best result presents an error of:

$$e = \sqrt{\frac{1}{K} \sum_{k=1}^K (\hat{y}_k - y_k)^2} \approx 2.85 \cdot 10^{-9}$$

Where K denotes the number of points (x_k, y_k) given to fit, and $\hat{y}_k$ denotes the value that our estimated function takes in the values x_k .

Decoding our best chromosome we obtain the following symbolic expression:

$$\hat{g}(x) = \frac{1 - x^2 + x^3}{1 - x^2 + x^4} \quad (8)$$

For the given shape:

$$g(x) = \frac{a_0 + a_1x + a_2x^2 + \dots + a_px^p}{b_0 + b_1x + b_2x^2 + \dots + b_qx^q} \quad (9)$$

we can use (8) to determine the constants. We can see how the order of the polynomial in the numerator is $p = 3$, while the polynomial in the denominator is of order $q = 4$. Also, $a_0 = 1, a_1 = 0, a_2 = -1, a_3 = 1; b_0 = 1, b_1 = 0, b_2 = -1, b_3 = 0, b_4 = 1$.

Figure 5 shows a plot of the data and the estimated function.

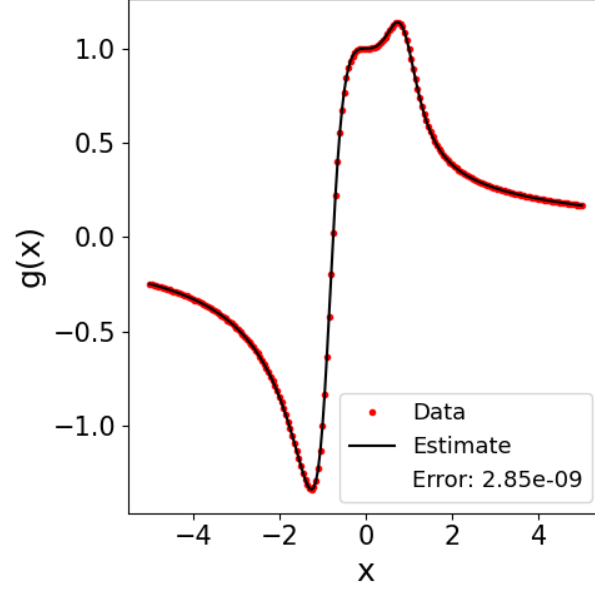


Figure 5: Plot of the data points in red, along with the estimated function $g(x)$ (8) in black. The rounded error is also presented in the legend.

Notes on the code: These exact results can be reproduced using the seed given in the file *run_lgp.py*. The presented results were obtained after $85 \cdot 10^3$ generations, but the same error was already achieved after $19 \cdot 10^3$ generations. When running the code *run_lgp.py*, if we press 's + Enter', we save the current best chromosome in the file *best_chromosome.py*; and whether we previously saved or not, the last best chromosome will be saved in this file.